

1) Read a dataset from the user and i. Use the Find-S algorithm to find the most specific hypothesis that is consistent with the positive examples. Ii. What is the final hypothesis after processing all the positive examples? Using the same dataset, apply the Candidate Elimination algorithm. Determine the final version space after processing all examples (both positive and negative). What are the most specific and most general hypotheses in the version space?

Program 1: Find-S Algorithm

```
import pandas as pd

# Read the dataset
data = pd.read_csv("EnjoySport.csv")

# Get attributes and target
concepts = data.iloc[:, :-1].values
target = data.iloc[:, -1].values

# Initialize hypothesis
hypothesis = None

# Apply Find-S Algorithm
for i in range(len(concepts)):
    if target[i] == "Yes":

        if hypothesis is None:
            hypothesis = list(concepts[i])

        else:
            for j in range(len(hypothesis)):
                if hypothesis[j] != concepts[i][j]:
                    hypothesis[j] = "?"
```

```
# Display the final hypothesis
print("Final Hypothesis:")
print(hypothesis)
```

Output:

Final Hypothesis:
['Sunny', 'Warm', '?', 'Strong', '?', '?']

Program 2: Candidate Elimination Algorithm

```
import pandas as pd

# Read dataset
data = pd.read_csv("EnjoySport.csv")

concepts = data.iloc[:, :-1].values
target = data.iloc[:, -1].values

# Initialize S and G
S = list(concepts[0])
G = [['?' for i in range(len(S))] for j in range(len(S))]

for i in range(len(concepts)):

    if target[i] == "Yes":

        # Generalize S
        for j in range(len(S)):
            if S[j] != concepts[i][j]:
                S[j] = '?'
                G[j][j] = '?'

    else:

        # Specialize G
```

```

    for j in range(len(S)):
        if S[j] != concepts[i][j]:
            G[j][j] = S[j]
        else:
            G[j][j] = '?'

# Remove empty hypotheses
G_final = []
for g in G:
    if g != ['?'] * len(S):
        G_final.append(g)

print("Specific Hypothesis (S):")
print(S)

print("\nGeneral Hypotheses (G):")
for g in G_final:
    print(g)

```

Output:

Specific Hypothesis (S):

['Sunny', 'Warm', '?', 'Strong', '?', '?']

General Hypotheses (G):

['Sunny', '?', '?', '?', '?', '?']

['?', 'Warm', '?', '?', '?', '?']

Dataset:

 jupyter enjoysport.csv Last Checkpoint: 11 months ago								
File Edit View Settings Help								
Delimiter: ,								
	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport	
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes	
2	Sunny	Warm	High	Strong	Warm	Same	Yes	
3	Rainy	Cold	High	Strong	Warm	Change	No	
4	Sunny	Warm	High	Strong	Cool	Change	Yes	

2) Read a supervised dataset and use bagging and boosting technique to classify the dataset. Indicate the performance of the model.

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingClassifier, AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# ---- Bagging ----
bagging = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=50,
    random_state=42
)
bagging.fit(X_train, y_train)
y_pred_bagging = bagging.predict(X_test)
bagging_acc = accuracy_score(y_test, y_pred_bagging)

# ---- Boosting (AdaBoost with SAMME to avoid warning) ----
boosting = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=50,
```

```

    algorithm="SAMME", # ✓ avoids FutureWarning
    random_state=42
)
boosting.fit(X_train, y_train)
y_pred_boosting = boosting.predict(X_test)
boosting_acc = accuracy_score(y_test, y_pred_boosting)

# ---- Results ----
print("Bagging Accuracy:", bagging_acc)
print("Boosting Accuracy:", boosting_acc)

print("\n--- Bagging Classification Report ---")
print(classification_report(y_test, y_pred_bagging,
target_names=iris.target_names))

print("\n--- Boosting Classification Report ---")
print(classification_report(y_test, y_pred_boosting,
target_names=iris.target_names))

```

Output:

Bagging Accuracy: 0.9333333333333333				
Boosting Accuracy: 0.9333333333333333				
--- Bagging Classification Report ---				
	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.88	0.93	0.90	15
virginica	0.93	0.87	0.90	15
accuracy			0.93	45
macro avg	0.93	0.93	0.93	45
weighted avg	0.93	0.93	0.93	45
--- Boosting Classification Report ---				
	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.88	0.93	0.90	15
virginica	0.93	0.87	0.90	15
accuracy			0.93	45
macro avg	0.93	0.93	0.93	45
weighted avg	0.93	0.93	0.93	45

3) Read a dataset and use an example-based method (such as RIPPER or CN2) to generate a set of classification rules . Apply the FOIL algorithm (First-Order Inductive Learner) to learn first-order rules for predicting.

Program:

CN2

```
import pandas as pd

# -----
# 1. Create the dataset
# -----

data = [
    ("sparrow", "bird", 1, 1),
    ("eagle", "bird", 1, 1),
    ("penguin", "bird", 0, 0),
    ("bat", "mammal", 1, 1),
    ("cat", "mammal", 0, 0),
    ("butterfly", "insect", 1, 1)
]

columns = ["name", "type", "has_wings", "can_fly"]

df = pd.DataFrame(data, columns=columns)

print("Dataset:")
print(df)

# -----
# 2. Generate CN2-like rules
# -----

print("\nCN2-like Classification Rules:")

for attribute in ["type", "has_wings"]:
```

```

for value in df[attribute].unique():

    # Select examples satisfying the condition
    subset = df[df[attribute] == value]

    # Check whether all selected examples are positive
    if subset["can_fly"].all():

        print(
            f"IF {attribute} = {value} "
            f"THEN can_fly = 1"
        )

```

Output:

Dataset:				
	name	type	has_wings	can_fly
0	sparrow	bird	1	1
1	eagle	bird	1	1
2	penguin	bird	0	0
3	bat	mammal	1	1
4	cat	mammal	0	0
5	butterfly	insect	1	1

CN2-like Classification Rules:				
IF type = insect THEN can_fly = 1				
IF has_wings = 1 THEN can_fly = 1				

FOIL

```

import pandas as pd
import math

# -----
# 1. Create the dataset
# -----

data = [
    ("sparrow", "bird", 1, 1),
    ("eagle", "bird", 1, 1),
    ("penguin", "bird", 0, 0),

```

```

("bat", "mammal", 1, 1),
("cat", "mammal", 0, 0),
("butterfly", "insect", 1, 1)
]
columns = ["name", "type", "has_wings", "can_fly"]
df = pd.DataFrame(data, columns=columns)

print("Dataset:")
print(df)
# -----
# 2. Create first-order facts
# -----
facts = {}
for _, row in df.iterrows():
    name = row["name"]
    facts[name] = set()
    # First-order predicate: type(X, value)
    facts[name].add(
        f"type({name},{row['type']})"
    )
    # First-order predicate: has_wings(X)
    if row["has_wings"] == 1:
        facts[name].add(
            f"has_wings({name})"
        )
# -----
# 3. Separate positive and negative examples
# -----
positive = []
negative = []

for _, row in df.iterrows():

```



```

    if row["can_fly"] == 1:
        positive.append(row["name"])
    else:
        negative.append(row["name"])
print("\nPositive Examples:")
print(positive)
print("\nNegative Examples:")
print(negative)

# -----
# 4. Check whether an example satisfies a literal
# -----

def satisfies(example, literal):

    fact = literal.replace("X", example)
    return fact in facts[example]

# -----
# 5. Find examples covered by a rule body
# -----

def covered(body, examples):
    result = []
    for example in examples:

        if all(
            satisfies(example, literal)
            for literal in body
        ):
            result.append(example)
    return result

# -----
# 6. Calculate FOIL information
# -----

def information_gain(p0, n0, p1, n1, t):

```

```

    if p1 == 0:
        return 0

    # Information before adding literal
    before = math.log2(
        p0 / (p0 + n0)
    )
    # Information after adding literal
    after = math.log2(
        p1 / (p1 + n1)
    )
    gain = t * (after - before)
    return gain

# -----
# 7. Calculate gain for a candidate literal
# -----
def calculate_gain(body, literal,
                   current_positive,
                   current_negative):

    # Number of positive examples before
    p0 = len(current_positive)

    # Number of negative examples before
    n0 = len(current_negative)

    # Add candidate literal
    new_body = body + [literal]

    # Positive examples covered after adding literal
    new_positive = covered(
        new_body,
        current_positive

```

```

)
# Negative examples covered after adding literal
new_negative = covered(
    new_body,
    current_negative
)
p1 = len(new_positive)
n1 = len(new_negative)

# t = number of positive examples
# covered after adding literal
t = p1
gain = information_gain(
    p0, n0, p1, n1, t
)
return gain, new_positive, new_negative
# -----
# 8. FOIL Algorithm
# -----
def foil(positive, negative):
    learned_rules = []
    # Positive examples not yet covered
    uncovered_positive = set(positive)
    while uncovered_positive:
        body = []
        current_positive = set(
            uncovered_positive
        )
        current_negative = set(
            negative
        )
        print("\n-----")
        print("Learning a new rule")

```

```

print("-----")
while current_negative:

    candidates = [
        "has_wings(X)",
        "type(X,bird)",
        "type(X,mammal)",
        "type(X,insect)"
    ]

    best_literal = None
    best_gain = float("-inf")

    print("\nCurrent rule body:", body)
    # -----
    # Evaluate every candidate literal
    # -----
    for literal in candidates:
        if literal in body:
            continue
        gain, new_positive, new_negative = \
            calculate_gain(
                body,
                literal,
                current_positive,
                current_negative
            )
        print(
            f"{literal:20s} "
            f"Gain = {gain:.4f}"
        )
        # Select literal with highest gain
        if gain > best_gain:

```

```
        best_gain = gain
        best_literal = literal

    # No useful literal found
    if best_literal is None:
        break

    # Add best literal to rule
    body.append(best_literal)

    # Update covered examples
    current_positive = set(
        covered(
            body,
            current_positive
        )
    )

    current_negative = set(
        covered(
            body,
            current_negative
        )
    )

    print(
        "\nSelected literal:",
        best_literal
    )

    print(
        "Positive covered:",
        current_positive
    )
```

```

        print(
            "Negative covered:",
            current_negative
        )

    # Save learned rule
    learned_rules.append(body)

    # Remove positive examples covered by rule
    uncovered_positive -= current_positive

    return learned_rules

# -----
# 9. Learn rules
# -----

rules = foil(
    positive,
    negative
)

# -----
# 10. Display final FOIL rules
# -----

print("\n\n=====")
print("FINAL FOIL RULES")
print("=====")

for i, rule in enumerate(rules, 1):
    print(
        f"Rule {i}: "
        f"can_fly(X) :- "
        f"{'', 'join(rule)}."
    )

```

Output:

```

Dataset:
      name  type  has_wings  can_fly
0  sparrow  bird         1         1
1   eagle  bird         1         1
2  penguin  bird         0         0
3     bat  mammal         1         1
4     cat  mammal         0         0
5 butterfly  insect         1         1

Positive Examples:
['sparrow', 'eagle', 'bat', 'butterfly']

Negative Examples:
['penguin', 'cat']

-----
Learning a new rule
-----

Current rule body: []
has_wings(X)      Gain = 2.3399
type(X,bird)      Gain = 0.0000
type(X,mammal)    Gain = -0.4150
type(X,insect)    Gain = 0.5850

Selected literal: has_wings(X)
Positive covered: {'bat', 'sparrow', 'butterfly', 'eagle'}
Negative covered: set()

=====
FINAL FOIL RULES
=====
Rule 1: can_fly(X) :- has_wings(X).

```

4) Read an unsupervised dataset and group the dataset based on similarity based on k-means clustering.

Program:

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris

# Step 1: Load the Iris dataset (ignore labels for unsupervised learning)
iris = load_iris()
data = pd.DataFrame(iris.data, columns=iris.feature_names)

```

Step 2: Apply K-Means clustering (3 clusters for 3 species)

```
kmeans = KMeans(n_clusters=3, random_state=42)
```

```
data['Cluster'] = kmeans.fit_predict(data)
```

Step 3: Print cluster centers

```
print("Cluster Centers:")
```

```
print(kmeans.cluster_centers_)
```

Step 4: Plot (using petal length vs petal width for clarity)

```
plt.scatter(data['petal length (cm)'], data['petal width (cm)'],
```

```
            c=data['Cluster'], cmap='viridis', s=50)
```

```
plt.scatter(kmeans.cluster_centers_[0, 2], kmeans.cluster_centers_[0, 3],
```

```
            c='red', marker='X', s=200, label='Centroids')
```

```
plt.xlabel("Petal length (cm)")
```

```
plt.ylabel("Petal width (cm)")
```

```
plt.title("K-Means Clustering on Iris Dataset")
```

```
plt.legend()
```

```
plt.show()
```

Output:

